CODEFLOW / PSUT

Industry Practices Using the SDLC

Session 01 Study Guide

A detailed explanation of every idea from the slides, written for beginners.

How to use this guide

This guide goes through every idea from Session 01 slowly, in plain language. Read it after class, or before the next session. Each section matches a part of the slides, so you can jump straight to what you want to review.

You do not need to memorise anything. The goal is to understand **why** things work the way they do. If a word is new, it is explained the first time it appears, and the glossary at the end collects them all.

Part	Topic	Slides
1	What is a software engineer?	2-3
2	A short history of computing	4-6
3	The SDLC: how software gets built	7-9
4	How the web works	10-17
5	The industry toolkit	18-20
6	HTML, CSS and JavaScript (overview)	21-25
7	Recap, homework and glossary	26-36

Part 1 - What is a software engineer?

> **Think first**
>
> What is a software engineer? Someone who writes code? Builds websites? Fixes bugs?

All three answers are partly right, but none is the whole story. Writing code is one **skill**. Being an engineer is a **way of working**.

> **Key idea**
>
> An engineer solves problems by applying science, maths and technical knowledge to **design, build, test and improve** solutions to real-world problems.

Engineers in other fields

Think about a civil engineer building a bridge. They do not just start laying bricks. They first understand what the bridge must do (carry 10,000 cars a day), design it, check the design on paper, build it, test it, and keep inspecting it for years. A mechanical engineer building an engine and an electrical engineer designing a circuit follow the same pattern.

Field	What they build	The same discipline
Civil	Bridges, roads, buildings	Understand the need, design, test, maintain
Mechanical	Engines, machines	Understand the need, design, test, maintain

Field	What they build	The same discipline
Electrical	Circuits, power systems	Understand the need, design, test, maintain
Software	Apps, websites, systems	Understand the need, design, test, maintain

Notice that the last column never changes. **Programming is a tool. Engineering is the discipline.** A good engineer also weighs *trade-offs*: a faster solution may cost more, a simpler design may be less flexible. Choosing well between options is a large part of the job.

> **Remember**
>
> Writing code is something an engineer does. Solving the right problem, in a reliable way, and keeping it working over time is what makes someone an engineer.

Part 2 - A short history of computing

> **Think first**
>
> How did we get here? Before apps, websites and AI, there were centuries of inventions.

Everything you use today stands on the work of people who solved hard problems with far less than you have. You do not need to memorise dates. Notice instead **what problem each person solved**, because you will use their ideas in this course.

The foundations (1843 - 1960s)

Who	When	What they did	Why it matters to you
Ada Lovelace	1843	Published the first algorithm meant for a machine (Babbage's proposed Analytical Engine).	An **algorithm** is a step-by-step recipe. Every program is one.
Alan Turing	1936	Described, on paper, what it means for something to be "computable".	The theory behind what computers can and cannot do.
The ENIAC Six	1940s	Six women who programmed ENIAC, one of the first electronic computers.	Programming began as careful, hands-on problem solving.
Grace Hopper	1950s	Built an early compiler, which turns human-readable instructions into machine instructions.	This is why we can write readable code instead of 1s and 0s.
John McCarthy	1956	Coined the term "artificial intelligence" and created the Lisp language.	The roots of the AI tools you use today.
Margaret Hamilton	1960s	Led the software team for the Apollo moon missions and helped establish "software engineering" as a discipline.	Software can be safety-critical, so it must be engineered, not just written.

Building the digital world (1969 onwards)

Who	When	What they did	Why it matters to you
Ritchie and Thompson	1969+	Created UNIX and the C language.	Most modern operating systems and languages descend from these.
Edgar F. Codd	1970	Invented the relational database model.	Databases like MySQL and PostgreSQL (Part 4) are built on it.
Barbara Liskov	1970s+	Pioneered data abstraction and modular design.	Hiding complexity behind simple pieces: the idea behind functions and components.
Radia Perlman	1980s	Invented the Spanning Tree Protocol for networks.	Helps networks stay reliable, so the internet does not loop endlessly.
Tim Berners-Lee	1989	Created the World Wide Web: HTML and HTTP.	**You will write HTML in this course.**
Linus Torvalds	1991 / 2005	Created the Linux kernel and the Git version control system.	**You will use Git and GitHub.**

Part 3 - The SDLC: how software gets built

> **Think first**
>
> How does an idea become real software? Hint: writing code is only one part of the process.

SDLC stands for **Software Development Life Cycle**. It is a structured way to build software that actually solves the problem. Without a process, teams build the wrong thing, find bugs too late, and step on each other's work.

Why have a process at all? Three reasons from the slides:

- **Clarity** - everyone knows what users need.
- **Quality** - problems are found early, when they are cheap to fix.
- **Teamwork** - work and releases are coordinated.

The six stages, one by one

We will follow one running example: **an online store**.

Stage	The question it answers	Online store example
1. Plan	What problem are we solving, and what must the software do? (These needs are called **requirements**.)	Interview shop owners and shoppers. Decide: customers must browse products, add to a cart, and pay securely.

Stage	The question it answers	Online store example
2. Design	How will it be organised and how will it feel to use? (**Architecture** = how the parts fit together. **UX** = user experience.)	Sketch the pages. Decide there will be a website, a server, and a database of products.
3. Develop	Let's build it.	Write the code for the product pages, the cart and the checkout.
4. Test	Does it behave correctly?	Try adding items, paying with a wrong card, and using a phone. Fix what breaks.
5. Deploy	How do users get it?	Put the site on a server so anyone can visit it.
6. Maintain	How do we keep it working and improving?	Fix new bugs, add discount codes, keep it secure and fast.

Key idea

Real projects revisit stages. After you deploy, user feedback sends you back to planning. Teams that work in an **Agile** way go around the cycle in small, frequent loops (for example every two weeks) instead of once, slowly. This is why the slide says: *six stages, one continuous cycle*.

Why test early?

Imagine finding out in the Maintain stage that the whole checkout idea was wrong. Fixing a mistake costs far more the later you catch it. That is the main reason for the SDLC.

Part 4 - How the web works

Think first

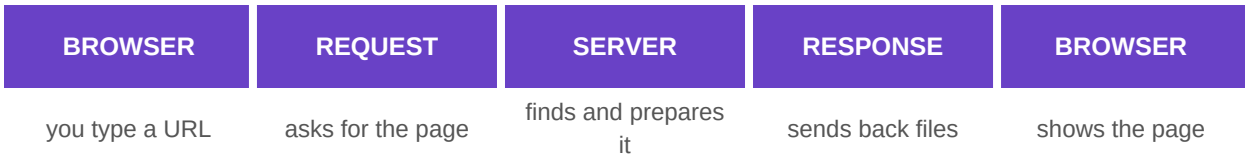
What actually happens when you open a website? Your browser is only one part of the system.

The internet and the web are not the same thing

- The **internet** is the physical and technical network: connected computers, cables, Wi-Fi.
- The **web** is a service that runs on the internet: linked pages and resources you open in a browser.

An analogy: the internet is the road system, the web is one kind of traffic on it (email and online games are other kinds).

The request and response cycle



1 You type a URL such as `codeflow.example` or click a link. The browser is the **client**.

2. The browser sends an **HTTP request** over the internet: "please give me this page".
3. A **server** (a computer that is always on and connected) receives the request and finds or builds the answer.
4. The server sends back a **response**: HTML, CSS and JavaScript files.
5. The browser reads those files and **renders** (draws) the page on your screen.

HTTP is simply the agreed language that browsers and servers use to talk to each other, like a polite way of asking and answering.

> **Think first**
>
> Why do some websites change for every user? Compare a portfolio with a social-media feed.

Static vs. dynamic websites

	Static	Dynamic
How it works	The server hands out prebuilt files. Everyone gets the same thing.	Content is created per request, based on who you are and on data.
Examples	A simple portfolio, a documentation page.	A social feed, a shopping cart, a banking app.
Built with	HTML + CSS + optional JavaScript.	Often servers, APIs and databases as well.

> **A common misunderstanding**
>
> **Static does not mean "no JavaScript".** A static site can still have menus, animations and buttons that react. Static only describes how the content is delivered: ready-made files, rather than pages assembled for each visitor.

> **Think first**
>
> Where does your code actually run? Your device? Someone else's computer? Both?

Client-side vs. server-side

	Client-side	Server-side
Where it runs	In the user's browser.	On a server.
Good for	Showing the interface, reacting to clicks, checking that a form is filled in, updating the page.	Logging people in, deciding who may do what, business rules, reading and saving data.
Can the user see it?	Yes, anyone can inspect it.	No, it stays on the server.

Login example. The browser sends your email and password to the server. The server checks them against its stored data and answers "yes" or "no". The browser then shows you your

account or an error. The check must happen on the server: if it happened in the browser, anyone could read and change the code to skip it.

> **Think first**
>
> What is behind an app like Instagram? The screen is only the visible layer.

Frontend, backend, database

Big apps are split into three parts with separate responsibilities:

Part	Job	Technologies on the slide	Instagram example
Frontend	What users see and touch.	HTML, CSS, JavaScript, React	The post and the like button.
Backend	The rules, services and APIs.	Node.js, Express, Python	Receives "user pressed like" and processes it.
Database	Stores the application's data.	PostgreSQL, MySQL, SQLite	Saves who liked which post.

An **API** (Application Programming Interface) is the doorway the frontend uses to ask the backend for things. Think of a waiter: you (frontend) order, the waiter (API) carries the request to the kitchen (backend), and the pantry (database) holds the ingredients.

When you press the like button: the frontend sends a request, the backend checks you are logged in and records the like, the database stores it, and the frontend updates the heart. All three cooperate in a fraction of a second.

Part 5 - The industry toolkit

> **Think first**
>
> Do engineers build everything from scratch? No. The best engineers know how to use the right tools.

Category	Examples	What it is for
Code editors	VS Code, IntelliJ	Where you write and organise code.
Version control	Git, GitHub, GitLab	Records every change, lets many people work together, and lets you undo mistakes. **Git** is the tool; **GitHub** is a website that hosts Git projects online.
Frameworks	React, Next.js, Express	Ready-made building blocks so you do not reinvent common features.
AI assistants	Claude, ChatGPT, Copilot	Explain, suggest and speed up work.
Design and tracking	Figma, Jira, Trello	Figma is for designing screens. Jira and Trello track tasks: who is doing what.
Deployment	Vercel, Netlify, AWS	Where your finished site lives so everyone can reach it.

> **About AI tools**
>
> AI tools accelerate work, but **you remain the engineer**. AI can be confidently wrong. You must read what it produces, test it, and take responsibility for the result. If you cannot explain a piece of code, you are not ready to ship it. Use AI to learn faster, not to skip learning.

From idea to production: tools meet the SDLC

PLAN	DESIGN	BUILD	TRACK	TEST	SHIP
Jira	Figma	VS Code	GitHub	DevTools	Vercel

Each SDLC stage has tools that help with it. **DevTools** are the testing and debugging panel built into your browser (press F12). Release is not the end: feedback and maintenance bring the team back to planning, so the cycle continues.

Part 6 - HTML, CSS and JavaScript

> **Think first**
>
> How does a browser know what to display? Every webpage starts with a structure.

Every website you have ever visited is built from three languages, each with one job. We can compare them to a person:

	HTML	CSS	JavaScript
Job	Structure	Appearance	Behavior
Like a person's...	Skeleton	Clothes and style	Movement
Controls	Headings, paragraphs, images, links, forms.	Colors, fonts, spacing, layout, responsiveness.	Reactions to clicks, interactivity, updating the page.

HTML = HyperText Markup Language. "Markup" means we *label* content: this is a heading, this is a paragraph, this is a button. **CSS** = Cascading Style Sheets. It says how the labelled things should look. **JavaScript** adds behavior, such as a menu that opens when clicked. Today we only started with HTML and CSS. JavaScript comes later.

The separate **HTML and CSS introduction handout** explains these two languages step by step, with examples to try. Start there before the practical.

Part 7 - The practical, recap and homework

The live workshop is to **build a developer profile**: a small personal webpage with your name and introduction, a profile image, skills or interests, a GitHub link, and your own colors.

The project in four steps

1. **Set up:** create a folder with an `index.html` file, a `style.css` file and an image. Open the folder in VS Code.

2. **HTML:** write the content first (structure).
3. **CSS:** style it second (colors, spacing, shapes). Save, then refresh the browser to see changes.
4. **Challenge (15 minutes):** change the colors, pick a font, add three skills, add a second link, try a hover effect.

Golden rules for beginners

1. **Change one thing, then refresh.** You will always know what caused a problem.
2. **Break things on purpose.** That is how you learn what each line does.
3. **Look carefully at what is wrong** before asking for help. Missing quotes and missing closing tags are the usual suspects.

Recap

- Engineering is more than writing code.
- Pioneers created the foundations of modern computing.
- The SDLC organises how teams deliver products: Plan, Design, Develop, Test, Deploy, Maintain.
- Frontend, backend and database have different jobs.
- HTML and CSS are your first building blocks.

Homework

Finish your profile and propose **one new feature** you would add (for example a dark mode, a project section or a contact form). Write it as a sentence: "Visitors should be able to ...". That sentence is your first requirement, the Plan stage of the SDLC.

Check your understanding

Try answering without looking back. Answers follow.

1. In your own words, what is the difference between programming and engineering?
2. Put these in order: Test, Plan, Deploy, Design, Maintain, Develop.
3. Why can a static website still be interactive?
4. Why should a password be checked on the server and not in the browser?
5. In Instagram, which part (frontend, backend, database) stores the information about your likes?
6. Which language is for structure, and which is for appearance?
7. What is Git, and how is it different from GitHub?

Answers

1. Programming is a tool for writing instructions. Engineering is the discipline of understanding a problem, designing, testing and maintaining a solution.

2. Plan, Design, Develop, Test, Deploy, Maintain.
3. Static means the files are prebuilt; JavaScript inside them can still react to the user.
4. Browser code can be viewed and changed by the user, so a check there can be bypassed.
5. The database stores it. The backend decides what to store, and the frontend shows it.
6. HTML is structure; CSS is appearance.
7. Git is the version-control tool on your computer; GitHub is an online service that hosts Git projects.

Glossary

Term	Meaning
Algorithm	A step-by-step set of instructions to solve a problem.
API	A defined way for one program to ask another for something.
Backend	The part of an app that runs on a server: rules, data access.
Browser	The program (Chrome, Edge, Safari...) that requests and displays web pages.
Client	The device or browser asking for something.
Compiler	A program that translates code into instructions a computer can run.
Database	Organised storage for an application's data.
Deploy	Release software so users can use it.
Frontend	The part users see and touch, running in the browser.
Git / GitHub	Version control tool / website that hosts Git projects.
HTTP	The language browsers and servers use to communicate.
Render	Turn code (HTML, CSS) into the visible page.
Requirements	What the software must do, from the users' needs.
SDLC	Software Development Life Cycle: the six stages of building software.
Server	A computer that answers requests, always on and connected.
URL	The address of a page, such as https://example.com/about.
UX	User experience: how it feels to use the product.
Version control	A system that records changes so you can go back and collaborate.